OPERATING SYSTEMS PROJECT

Linux Command Execution Time Profiler

Measuring Execution Time and Resource Utilization of Linux Commands

Section: 13
Team no: 22

G. Bhupathi - 2520090027
B. Siddhartha - 2500090225
G.V.Karthik reddy - 2520080006

How a command becomes a process

The shell parses input, requests a process, and hands execution to the kernel. The scheduler allocates CPU time while memory and I/O are accounted for.

WALL TIME	CPU	MEMORY
elapsed	utilization	peak RSS

Measure all three to explain system performance—not just speed.



The measurement gap

Manual estimates obscure what the operating system is actually doing.

PROBLEM

Create a tool that measures execution time and resource usage of Linux commands.

WHY IT MATTERS

Manual timing is inconsistent. Slow commands and resource-heavy processes are difficult to identify or compare reliably.

Need: one repeatable, automated measurement path

Objectives — with a realistic scope

A lightweight CLI instrument for learning, measurement, and comparison.

01

Precise elapsed time

02

CPU + peak memory

03

Arbitrary commands

04

Readable reports

05

Run comparison

06

OS learning

SCOPE • Undergraduate command-line tool; not a production observability platform.

One command in. One report out.

The profiler wraps execution, collects resource statistics, then returns a compact result.




```
$ profiler "sleep 2"

Command       sleep 2
Elapsed time  ~2.00 s
Exit status   0
CPU usage     collected during execution
Peak memory   resident set size
```

Illustrative output only
Values shown are not claimed measurements.

System architecture

A linear profiling path supported by Linux process and resource services.



SHELL · parsing, arguments, standard streams

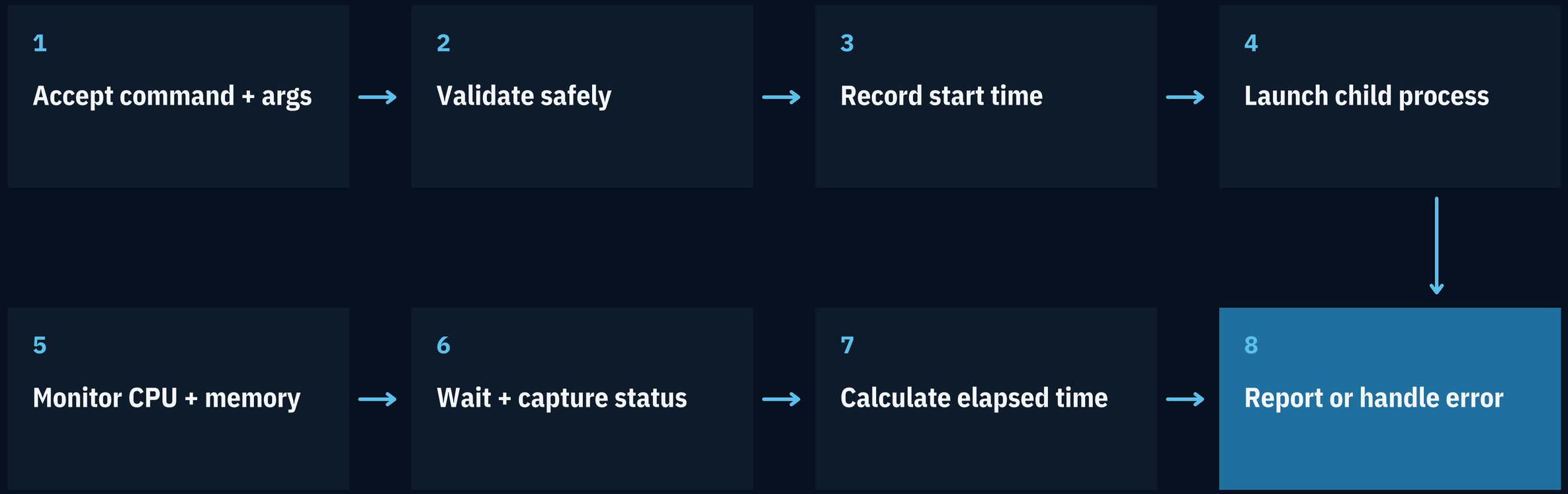
KERNEL · system calls, process accounting, timers

SCHEDULER · CPU allocation

RESOURCE MANAGEMENT · memory + I/O

Methodology: instrument the lifecycle

Monotonic timing brackets execution; monitoring runs while the child process is active.



OS concepts — applied, not abstract

Each concept maps directly to a profiler responsibility.

User ⇔ kernel

Shell input becomes system calls

Process control

Create, wait, terminate, read status

Scheduling + clocks

Measure wall time amid CPU scheduling

Resource accounting

Sample CPU time and resident memory

File descriptors

Capture stdin / stdout / stderr

Execution errors

Surface invalid command, timeout, signal

Test across distinct workloads

Illustrative placeholders only; values vary with collection method and kernel behavior.

```
** (gedit:495): WARNING **: 04:42:59.972: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', versi
on '1.0' not found
sidhartha@Siddhartha:~/OShackthon$ gcc profiler.c -o profiler
sidhartha@Siddhartha:~/OShackthon$ ./profiler

=========================================
 LINUX COMMAND EXECUTION TIME PROFILER
=========================================
Enter Linux commands or:
parallel    - Demonstrate concurrency
signal-demo - Demonstrate signals
exit        - Close profiler

profiler> ls -l
total 32
-rw-r--r-- 1 sidhartha sidhartha    49 Oct 16 14:53 execution_report.csv
-rwxr-xr-x 1 sidhartha sidhartha 17272 Oct 17 04:43 profiler
-rw-r--r-- 1 sidhartha sidhartha  7059 Oct 17 04:43 profiler.c

========== EXECUTION REPORT ==========
Command       : ls -l
Process ID    : 1688
Execution Time: 0.006886 seconds
Exit Status   : 0
Result        : SUCCESS
Peak Child Memory: 8700 KB
======================================

profiler>
```

Course outcomes + references

The project connects Linux mechanisms to measurable process behavior.

- CO-1 Service layer + system calls
- CO-2 Process control + lifecycle
- CO-3 stdout / stderr capture
- CO-4 RSS memory monitoring
- CO-5 file descriptors + I/O
- CO-6 concurrent profiling extension

REFERENCES

Linux man pages
time(1)
getrusage(2)
procfs documentation
Python subprocess documentation
GNU time utility documentation


```
$ exit 0
```

From OS theory to observable behavior

A small profiler makes process execution, timing, and resource accounting tangible.

The tool measures elapsed time, CPU use, memory use, and exit status in one repeatable command-line workflow.

NEXT ITERATION

Multi-command comparison
CSV report export
Historical trends
CLI or GUI charts
Timeouts + repeated benchmarks

Measure → explain → improve